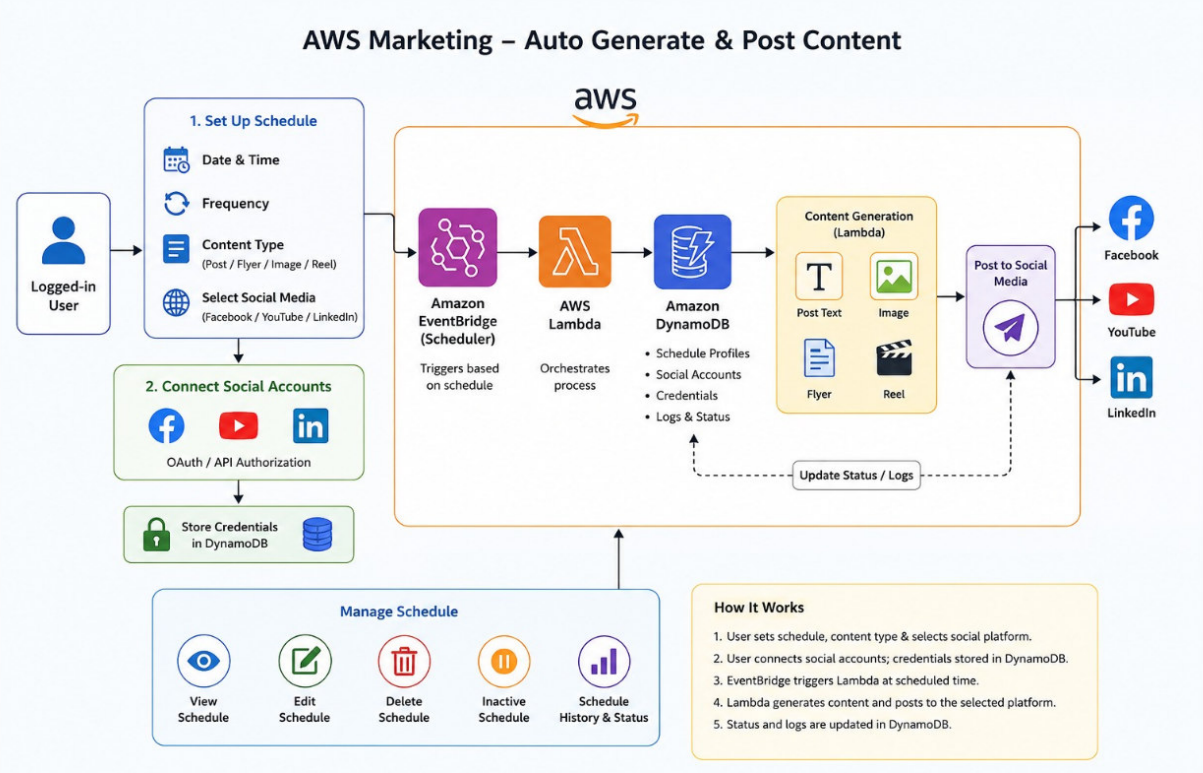
Lab: AWS Marketing Automation

Auto-Generate and Post Social Media Content with EventBridge Scheduler, Lambda, DynamoDB, API Gateway, and AWS Amplify



Field	Value
Lab Title	AWS Marketing Automation - Auto-Generate & Post Content
Level	Intermediate
Estimated Time	3 - 4 hours
Region	us-east-1
AWS Services	DynamoDB, Lambda, EventBridge Scheduler, IAM, API Gateway (HTTP API), Amplify Hosting, CloudWatch Logs
Languages / Tools	Python 3.12 (boto3), AWS CLI v2, Node.js + React (frontend), pytest + moto (tests)

1. Objective

In this lab you build a serverless marketing automation platform. A logged-in user connects social media accounts (Facebook, YouTube, LinkedIn), defines a content schedule (post / flyer / image / reel, time, frequency, timezone), and the system automatically generates the content and posts it to the selected platform at the scheduled time.

By the end of this lab you will be able to:

- Create DynamoDB tables programmatically with boto3 to store social connections, schedule profiles, and execution logs.
- Create IAM roles that let Lambda access DynamoDB and EventBridge Scheduler, and let EventBridge Scheduler invoke Lambda.
- Build a Worker Lambda that generates content and posts it to a social platform, and a Schedule Manager Lambda that creates, views, updates, inactivates, and deletes schedules.
- Wire EventBridge Scheduler to invoke the Worker Lambda on cron or one-time schedules with exact timing (FlexibleTimeWindow = OFF).
- Expose the Schedule Manager through API Gateway (HTTP API) so an Amplify-hosted React frontend can drive the whole workflow.
- Verify everything with unit tests (pytest + moto) and integration tests (live Lambda invokes and API Gateway calls).

2. Architecture Overview

Flow: **Amplify frontend** → **API Gateway (HTTP API)** → **Schedule Manager Lambda** → writes to **DynamoDB** and creates an **EventBridge schedule**. At the scheduled time, **EventBridge Scheduler** → invokes **Worker Lambda** → reads schedule + credentials from DynamoDB → generates content → posts to the social platform → writes status to **ScheduleLogs**.

Component	Responsibility
Amplify frontend (React)	UI for connecting accounts and managing schedules; calls the API
API Gateway (HTTP API)	Public HTTPS endpoint with CORS in front of the Schedule Manager Lambda
MarketingScheduleManager (Lambda)	CRUD for social connections and schedules; creates/updates/deletes EventBridge schedules
Amazon EventBridge Scheduler	Fires at the exact scheduled time and invokes the Worker Lambda with the schedule_id
MarketingContentWorker (Lambda)	Generates content, posts to Facebook/YouTube/LinkedIn, writes logs and status
DynamoDB	SocialConnections, ContentSchedules, ScheduleLogs tables
CloudWatch Logs	Execution logs for both Lambdas

Note: This lab uses Amazon EventBridge Scheduler (not EventBridge Rules and not Azure Event Grid). Scheduler supports one-time at() expressions, cron() with timezones, and direct Lambda targets.

2.1 Key Resource Names (used throughout the lab)

Every command in this lab refers to these exact names. Keep this list handy:

Lambda functions: `MarketingContentWorker` (posts content) • `MarketingScheduleManager` (schedule CRUD API)

DynamoDB tables: `SocialConnections` `ContentSchedules` `ScheduleLogs`

IAM roles: `MarketingAutomationLambdaRole` `EventBridgeSchedulerInvokeLambdaRole`

API / App: `MarketingAutomationAPI` (HTTP API) `marketing-frontend` (Amplify app)

2.2 End-to-End Flow

This is the complete journey of one scheduled post. Each stage maps to a lab step, so you can trace any failure to the step that built that stage:

1. The user opens the **Amplify frontend** (Step 8), picks platform, content type, topic, date/time, frequency, and timezone.
2. The browser sends the form as JSON to **API Gateway** (Step 7) over HTTPS.
3. API Gateway proxies the request to `MarketingScheduleManager` (Step 6).
4. The manager saves the profile to `ContentSchedules` and creates an **EventBridge schedule** named marketing-<schedule_id> targeting the Worker (Steps 2, 4, 6).
5. At the scheduled time, **EventBridge Scheduler** assumes its invoke role and calls `MarketingContentWorker` with {"schedule_id": "..."} (Steps 4, 5).
6. The Worker loads the schedule from `ContentSchedules` and the user's credentials from `SocialConnections` (Steps 2, 5).
7. The Worker generates the content (post / flyer / image / reel) and posts it to Facebook, YouTube, or LinkedIn (Step 5).
8. The Worker updates last_run_status on the schedule and writes a success / failed / skipped entry to `ScheduleLogs` (Step 5).
9. The user views run history via view_schedule in the UI, and you verify it with the tests (Steps 9, 13, 14).

Flow stage	Built and verified in
Amplify frontend UI	Step 8 (build + deploy), Step 12 checklist
API Gateway endpoint	Step 7 (create + curl smoke test)
Schedule Manager Lambda	Step 6 (deploy), Step 9 (functional tests), unit tests
DynamoDB tables	Step 2 (create), verified in every functional test
EventBridge schedule	Step 6 (created by manager), Step 9.2/9.3 (verify)
Worker Lambda run	Step 5 (deploy), Step 9.6 (manual invoke), integration test 03
Logs & status	Step 9.7 (ScheduleLogs scan), integration test 03

3. Prerequisites

- An AWS account with permission to create IAM roles, Lambda functions, DynamoDB tables, EventBridge schedules, API Gateway APIs, and Amplify apps.
- AWS CLI v2 installed and configured (`aws configure`) with region `us-east-1`.
- Python 3.12 with boto3 installed (pip install boto3).
- Node.js 18+ and npm (for the React frontend).
- Your AWS account ID. Find it with: `aws sts get-caller-identity --query Account --output text`. Everywhere this lab says YOUR_ACCOUNT_ID, substitute that value.

Note: AWS CLI v2 requires --cli-binary-format raw-in-base64-out when passing JSON payloads to aws lambda invoke. All invoke commands in this lab include it. If you omit it you will get an 'Invalid base64' error.

4. Step 1 - Project Setup

WHAT WE ARE DOING

Create a local project folder that will hold the table-creation script, both Lambda functions, IAM policy documents, tests, and the frontend.

WHY

A consistent layout keeps packaging commands (zip, deploy) simple and makes the unit/integration tests at the end of the lab easy to run.

▶ RUN / CODE

```
mkdir aws-marketing-automation
cd aws-marketing-automation
mkdir worker_lambda schedule_manager_lambda tests frontend
touch create_tables.py
```

✓ EXPECTED OUTPUT

```
aws-marketing-automation/
├── create_tables.py
├── worker_lambda/
├── schedule_manager_lambda/
├── tests/
└── frontend/
```

5. Step 2 - Create DynamoDB Tables

Resources in this step: `SocialConnections` `ContentSchedules` `ScheduleLogs`

WHAT WE ARE DOING

Create three DynamoDB tables with a Python script: SocialConnections (composite key user_id + platform), ContentSchedules (key schedule_id), and ScheduleLogs (key log_id).

WHY

SocialConnections stores OAuth credentials per user per platform - the composite key lets one user connect many platforms. ContentSchedules is the schedule profile the Worker Lambda reads at run time. ScheduleLogs is an append-only audit trail (success / failed / skipped) so users can see schedule history and status. PAY_PER_REQUEST billing avoids capacity planning for a lab.

5.1 Create create_tables.py

▶ **RUN / CODE**

```
import boto3
from botocore.exceptions import ClientError

REGION = "us-east-1"

dynamodb = boto3.client("dynamodb", region_name=REGION)


def create_table(table_name, key_schema, attribute_definitions):
    try:
        response = dynamodb.create_table(
            TableName=table_name,
            KeySchema=key_schema,
            AttributeDefinitions=attribute_definitions,
            BillingMode="PAY_PER_REQUEST"
        )

        print(f"Creating table: {table_name}")
        waiter = dynamodb.get_waiter("table_exists")
        waiter.wait(TableName=table_name)
        print(f"Table created successfully: {table_name}")
        return response

    except ClientError as error:
        if error.response["Error"]["Code"] == "ResourceInUseException":
            print(f"Table already exists: {table_name}")
```

```
        else:
            raise error


def main():
    create_table(
        table_name="SocialConnections",
        key_schema=[
            {"AttributeName": "user_id", "KeyType": "HASH"},
            {"AttributeName": "platform", "KeyType": "RANGE"}
        ],
        attribute_definitions=[
            {"AttributeName": "user_id", "AttributeType": "S"},
            {"AttributeName": "platform", "AttributeType": "S"}
        ]
    )

    create_table(
        table_name="ContentSchedules",
        key_schema=[
            {"AttributeName": "schedule_id", "KeyType": "HASH"}
        ],
        attribute_definitions=[
            {"AttributeName": "schedule_id", "AttributeType": "S"}
        ]
    )

    create_table(
        table_name="ScheduleLogs",
        key_schema=[
            {"AttributeName": "log_id", "KeyType": "HASH"}
        ],
        attribute_definitions=[
            {"AttributeName": "log_id", "AttributeType": "S"}
        ]
    )


if __name__ == "__main__":
    main()
```

5.2 Run and validate

▶ **RUN / CODE**

```
python create_tables.py
aws dynamodb list-tables --region us-east-1
```

✓ EXPECTED OUTPUT

```
Creating table: SocialConnections
Table created successfully: SocialConnections
Creating table: ContentSchedules
Table created successfully: ContentSchedules
Creating table: ScheduleLogs
Table created successfully: ScheduleLogs

{
    "TableNames": [
        "ContentSchedules",
        "ScheduleLogs",
        "SocialConnections"
    ]
}
```

6. Step 3 - IAM Role for the Lambda Functions

WHAT WE ARE DOING
Create the MarketingAutomationLambdaRole that both Lambdas will assume, attach the AWS-managed basic execution policy (CloudWatch logging), and add an inline policy for DynamoDB, EventBridge Scheduler, and iam:PassRole.

WHY
Lambda cannot touch any AWS resource without an execution role. The Schedule Manager needs scheduler:* actions to create/update/delete schedules, and it needs iam:PassRole because when it creates a schedule it hands EventBridge Scheduler a role to invoke the Worker Lambda with - AWS requires explicit permission to pass a role to another service.

6.1 Trust policy - lambda-trust-policy.json

▶ **RUN / CODE**

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "lambda.amazonaws.com"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

6.2 Create the role and attach basic execution

▶ **RUN / CODE**

```
aws iam create-role \
  --role-name MarketingAutomationLambdaRole \
  --assume-role-policy-document file://lambda-trust-policy.json

aws iam attach-role-policy \
  --role-name MarketingAutomationLambdaRole \
  --policy-arn arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole
```

6.3 Inline policy - lambda-dynamodb-scheduler-policy.json

Replace YOUR_ACCOUNT_ID before saving.

▶ **RUN / CODE**

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "dynamodb:GetItem",
        "dynamodb:PutItem",
        "dynamodb:UpdateItem",
        "dynamodb:DeleteItem",
        "dynamodb:Scan",
        "dynamodb:Query"
      ],
      "Resource": [
        "arn:aws:dynamodb:us-east-1:YOUR_ACCOUNT_ID:table/SocialConnections",
        "arn:aws:dynamodb:us-east-1:YOUR_ACCOUNT_ID:table/ContentSchedules",
        "arn:aws:dynamodb:us-east-1:YOUR_ACCOUNT_ID:table/ScheduleLogs"
      ]
    },
    {
      "Effect": "Allow",
      "Action": [
        "scheduler:CreateSchedule",
        "scheduler:UpdateSchedule",
        "scheduler:DeleteSchedule",
        "scheduler:GetSchedule"
      ],
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": "iam:PassRole",
      "Resource":
"arn:aws:iam::YOUR_ACCOUNT_ID:role/EventBridgeSchedulerInvokeLambdaRole"
    }
  ]
}
```

▶ **RUN / CODE**

```
aws iam put-role-policy \
  --role-name MarketingAutomationLambdaRole \
  --policy-name MarketingAutomationLambdaPolicy \
  --policy-document file://lambda-dynamodb-scheduler-policy.json
```

6.4 Validate

▶ **RUN / CODE**

```
aws iam list-role-policies --role-name MarketingAutomationLambdaRole
```

✓ EXPECTED OUTPUT

```
{
    "PolicyNames": [
        "MarketingAutomationLambdaPolicy"
    ]
}
```

7. Step 4 - IAM Role for EventBridge Scheduler

WHAT WE ARE DOING
Create EventBridgeSchedulerInvokeLambdaRole, trusted by scheduler.amazonaws.com, allowed only to invoke the Worker Lambda.

WHY
When a schedule fires, EventBridge Scheduler assumes this role to call lambda:InvokeFunction on MarketingContentWorker. Scoping the resource to that single function ARN follows least privilege - the scheduler cannot invoke anything else.

7.1 Trust policy - scheduler-trust-policy.json

▶ **RUN / CODE**

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "scheduler.amazonaws.com"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

7.2 Permission policy - scheduler-invoke-lambda-policy.json

▶ **RUN / CODE**

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "lambda:InvokeFunction",
      "Resource": "arn:aws:lambda:us-east-
1:YOUR_ACCOUNT_ID:function:MarketingContentWorker"
    }
  ]
}
```

7.3 Create role and attach policy

▶ RUN / CODE

```
aws iam create-role \
  --role-name EventBridgeSchedulerInvokeLambdaRole \
  --assume-role-policy-document file://scheduler-trust-policy.json

aws iam put-role-policy \
  --role-name EventBridgeSchedulerInvokeLambdaRole \
  --policy-name SchedulerInvokeLambdaPolicy \
  --policy-document file://scheduler-invoke-lambda-policy.json
```

✓ EXPECTED OUTPUT

```
{
    "Role": {
        "RoleName": "EventBridgeSchedulerInvokeLambdaRole",
        "Arn":
"arn:aws:iam::YOUR_ACCOUNT_ID:role/EventBridgeSchedulerInvokeLambdaRole",
        ...
    }
}
```

8. Step 5 - Worker Lambda (Generate and Post Content)

Resources in this step: `MarketingContentWorker` `ContentSchedules` `SocialConnections` `ScheduleLogs`

WHAT WE ARE DOING

Create MarketingContentWorker. EventBridge Scheduler invokes it with {"schedule_id": "..."}. It loads the schedule profile and the user's social connection from DynamoDB, generates content for the requested type (post / flyer / image / reel), posts to the selected platform, updates the schedule's last_run_status, and writes a ScheduleLogs entry.

WHY

This is the execution engine of the system. Keeping it stateless (everything it needs comes from the event and DynamoDB) means any schedule can invoke it concurrently. The try/except wrapper guarantees a failed run still writes a 'failed' log and re-raises so the error is visible in CloudWatch and Lambda metrics. Facebook posting shows a real Graph API call pattern; LinkedIn and YouTube are mocked so the lab runs without live credentials - replace them with real API calls later. The create_content function is the hook for Amazon Bedrock or your own generation service.

8.1 Create worker_lambda/lambda_function.py

▶ **RUN / CODE**

```
import json
import uuid
import boto3
import datetime
import urllib.request
import urllib.parse
from botocore.exceptions import ClientError

REGION = "us-east-1"

dynamodb = boto3.resource("dynamodb", region_name=REGION)

schedules_table = dynamodb.Table("ContentSchedules")
connections_table = dynamodb.Table("SocialConnections")
logs_table = dynamodb.Table("ScheduleLogs")
```

```
def now_iso():
    return datetime.datetime.utcnow().isoformat()


def get_schedule(schedule_id):
    response = schedules_table.get_item(
        Key={"schedule_id": schedule_id}
    )
    return response.get("Item")


def get_social_connection(user_id, platform):
    response = connections_table.get_item(
        Key={
            "user_id": user_id,
            "platform": platform
        }
    )
    return response.get("Item")


def create_content(content_type, topic):
    """
    Replace this placeholder with your AI/image/video generation logic.
    Example integrations:
    - Amazon Bedrock for text/image generation
    - Your own image generation API
    - Your flyer/reel creation service
    """

    if content_type == "post":
        return {
            "type": "post",
            "text": f"Check out our latest update: {topic}. Shop now and
discover something unique!"
        }

    if content_type == "flyer":
        return {
            "type": "flyer",
            "text": f"Promotional flyer for {topic}",
            "media_url": "https://example.com/generated-flyer.png"
        }

    if content_type == "image":
        return {
            "type": "image",
            "text": f"New image campaign for {topic}",
            "media_url": "https://example.com/generated-image.png"
```

```
        }

    if content_type == "reel":
        return {
            "type": "reel",
            "text": f"New reel campaign for {topic}",
            "media_url": "https://example.com/generated-reel.mp4"
        }

    raise ValueError(f"Unsupported content type: {content_type}")


def post_to_facebook(connection, content):
    """
    Placeholder Facebook posting logic.
    Replace with real Facebook Graph API endpoint.
    """

    access_token = connection["access_token"]
    page_id = connection.get("page_id")

    if not page_id:
        raise ValueError("Facebook page_id is missing")

    url = f"https://graph.facebook.com/v20.0/{page_id}/feed"

    payload = {
        "message": content.get("text", ""),
        "access_token": access_token
    }

    encoded_payload = urllib.parse.urlencode(payload).encode("utf-8")

    request = urllib.request.Request(url, data=encoded_payload, method="POST")

    with urllib.request.urlopen(request) as response:
        return json.loads(response.read().decode("utf-8"))


def post_to_linkedin(connection, content):
    """
    Placeholder LinkedIn posting logic.
    Replace with LinkedIn API implementation.
    """

    return {
        "status": "mock_success",
        "platform": "linkedin",
```

```
        "message": "LinkedIn post simulated successfully"
    }


def post_to_youtube(connection, content):
    """
    Placeholder YouTube posting logic.
    Replace with YouTube upload API implementation.
    Usually used for reels/videos, not plain text posts.
    """

    return {
        "status": "mock_success",
        "platform": "youtube",
        "message": "YouTube upload simulated successfully"
    }


def post_to_social(platform, connection, content):
    platform = platform.lower()

    if platform == "facebook":
        return post_to_facebook(connection, content)

    if platform == "linkedin":
        return post_to_linkedin(connection, content)

    if platform == "youtube":
        return post_to_youtube(connection, content)

    raise ValueError(f"Unsupported platform: {platform}")


def write_log(schedule_id, user_id, platform, status, message,
response_data=None):
    log_id = str(uuid.uuid4())

    logs_table.put_item(
        Item={
            "log_id": log_id,
            "schedule_id": schedule_id,
            "user_id": user_id,
            "platform": platform,
            "status": status,
            "message": message,
            "response_data": response_data or {},
            "created_at": now_iso()
        }
    )
```

```
    return log_id


def update_schedule_status(schedule_id, status):
    schedules_table.update_item(
        Key={"schedule_id": schedule_id},
        UpdateExpression="SET last_run_status = :status, last_run_at =
:last_run_at",
        ExpressionAttributeValues={
            ":status": status,
            ":last_run_at": now_iso()
        }
    )


def lambda_handler(event, context):
    print("Received event:", json.dumps(event))

    schedule_id = event.get("schedule_id")

    if not schedule_id:
        raise ValueError("schedule_id is required")

    try:
        schedule = get_schedule(schedule_id)

        if not schedule:
            raise ValueError(f"Schedule not found: {schedule_id}")

        if schedule.get("status") != "active":
            write_log(
                schedule_id=schedule_id,
                user_id=schedule.get("user_id", "unknown"),
                platform=schedule.get("platform", "unknown"),
                status="skipped",
                message="Schedule is inactive"
            )
            return {
                "statusCode": 200,
                "body": "Schedule inactive. Skipped."
            }

        user_id = schedule["user_id"]
        platform = schedule["platform"]
        content_type = schedule["content_type"]
        topic = schedule.get("topic", "new promotion")

        connection = get_social_connection(user_id, platform)
```

```
        if not connection:
            raise ValueError(f"Social connection not found for {user_id} -
{platform}")

        content = create_content(content_type, topic)

        post_response = post_to_social(platform, connection, content)

        update_schedule_status(schedule_id, "success")

        write_log(
            schedule_id=schedule_id,
            user_id=user_id,
            platform=platform,
            status="success",
            message="Content generated and posted successfully",
            response_data=post_response
        )

        return {
            "statusCode": 200,
            "body": json.dumps({
                "message": "Success",
                "schedule_id": schedule_id,
                "platform": platform,
                "content_type": content_type
            })
        }

    except Exception as error:
        print("Error:", str(error))

        try:
            update_schedule_status(schedule_id, "failed")
            write_log(
                schedule_id=schedule_id,
                user_id="unknown",
                platform="unknown",
                status="failed",
                message=str(error)
            )
        except Exception as log_error:
            print("Failed to write log:", str(log_error))

        raise error
```

8.2 Package and deploy

▶ RUN / CODE

```
cd worker_lambda
zip -r worker_lambda.zip lambda_function.py

aws lambda create-function \
  --function-name MarketingContentWorker \
  --runtime python3.12 \
  --role arn:aws:iam::YOUR_ACCOUNT_ID:role/MarketingAutomationLambdaRole \
  --handler lambda_function.lambda_handler \
  --zip-file fileb://worker_lambda.zip \
  --timeout 60 \
  --memory-size 512 \
  --region us-east-1
cd ..
```

8.3 Validate

▶ RUN / CODE

```
aws lambda get-function \
  --function-name MarketingContentWorker \
  --region us-east-1
```

✓ EXPECTED OUTPUT

```
{
    "Configuration": {
        "FunctionName": "MarketingContentWorker",
        "Runtime": "python3.12",
        "State": "Active",
        ...
    }
}
```

9. Step 6 - Schedule Manager Lambda

Resources in this step: `MarketingScheduleManager` `ContentSchedules` `SocialConnections`

WHAT WE ARE DOING

Create MarketingScheduleManager. It is an action router: connect_social, create_schedule, view_schedule, update_schedule, inactive_schedule, delete_schedule. On create it writes the profile to ContentSchedules and creates a matching EventBridge schedule targeting the Worker Lambda; on update/inactive/delete it keeps DynamoDB and EventBridge Scheduler in sync.

WHY

The frontend needs one API surface for the entire schedule lifecycle. Keeping DynamoDB (the source of truth users see) and EventBridge Scheduler (the thing that actually fires) in sync in a single function prevents orphaned schedules. FlexibleTimeWindow Mode=OFF is set on every scheduler call because exact posting time matters for marketing content. Inactivation both flips the DynamoDB status to inactive AND disables the EventBridge schedule (State=DISABLED) - two layers of defense; even if the schedule fired, the Worker skips inactive profiles.

9.1 Create schedule_manager_lambda/lambda_function.py

Replace YOUR_ACCOUNT_ID at the top of the file.

▶ RUN / CODE

```
import json
import uuid
import boto3
import datetime
from botocore.exceptions import ClientError

REGION = "us-east-1"
ACCOUNT_ID = "YOUR_ACCOUNT_ID"

WORKER_LAMBDA_ARN =
f"arn:aws:lambda:{REGION}:{ACCOUNT_ID}:function:MarketingContentWorker"
SCHEDULER_ROLE_ARN =
f"arn:aws:iam::{ACCOUNT_ID}:role/EventBridgeSchedulerInvokeLambdaRole"

dynamodb = boto3.resource("dynamodb", region_name=REGION)
scheduler = boto3.client("scheduler", region_name=REGION)

schedules_table = dynamodb.Table("ContentSchedules")
connections_table = dynamodb.Table("SocialConnections")
```

```
def now_iso():
    return datetime.datetime.utcnow().isoformat()


def create_social_connection(body):
    required = ["user_id", "platform", "access_token"]

    for field in required:
        if field not in body:
            raise ValueError(f"{field} is required")

    item = {
        "user_id": body["user_id"],
        "platform": body["platform"].lower(),
        "access_token": body["access_token"],
        "refresh_token": body.get("refresh_token", ""),
        "page_id": body.get("page_id", ""),
        "channel_id": body.get("channel_id", ""),
        "organization_id": body.get("organization_id", ""),
        "status": "active",
        "created_at": now_iso(),
        "updated_at": now_iso()
    }

    connections_table.put_item(Item=item)

    return {
        "message": "Social connection saved",
        "user_id": item["user_id"],
        "platform": item["platform"]
    }


def create_schedule(body):
    required = [
        "user_id",
        "platform",
        "content_type",
        "schedule_expression",
        "topic"
    ]

    for field in required:
        if field not in body:
            raise ValueError(f"{field} is required")

    schedule_id = str(uuid.uuid4())
```

```
    schedule_name = f"marketing-{schedule_id}"

    item = {
        "schedule_id": schedule_id,
        "schedule_name": schedule_name,
        "user_id": body["user_id"],
        "platform": body["platform"].lower(),
        "content_type": body["content_type"].lower(),
        "topic": body["topic"],
        "schedule_expression": body["schedule_expression"],
        "timezone": body.get("timezone", "America/Chicago"),
        "status": "active",
        "last_run_status": "never_run",
        "created_at": now_iso(),
        "updated_at": now_iso()
    }

    schedules_table.put_item(Item=item)

    scheduler.create_schedule(
        Name=schedule_name,
        ScheduleExpression=body["schedule_expression"],
        ScheduleExpressionTimezone=item["timezone"],
        FlexibleTimeWindow={
            "Mode": "OFF"
        },
        Target={
            "Arn": WORKER_LAMBDA_ARN,
            "RoleArn": SCHEDULER_ROLE_ARN,
            "Input": json.dumps({
                "schedule_id": schedule_id
            })
        },
        State="ENABLED"
    )

    return {
        "message": "Schedule created",
        "schedule_id": schedule_id,
        "schedule_name": schedule_name
    }


def view_schedule(body):
    schedule_id = body.get("schedule_id")

    if not schedule_id:
        raise ValueError("schedule_id is required")
```

```
    response = schedules_table.get_item(
        Key={"schedule_id": schedule_id}
    )

    item = response.get("Item")

    if not item:
        return {
            "message": "Schedule not found"
        }

    return item


def update_schedule(body):
    schedule_id = body.get("schedule_id")

    if not schedule_id:
        raise ValueError("schedule_id is required")

    response = schedules_table.get_item(
        Key={"schedule_id": schedule_id}
    )

    item = response.get("Item")

    if not item:
        raise ValueError("Schedule not found")

    schedule_name = item["schedule_name"]

    platform = body.get("platform", item["platform"]).lower()
    content_type = body.get("content_type", item["content_type"]).lower()
    topic = body.get("topic", item["topic"])
    schedule_expression = body.get("schedule_expression",
item["schedule_expression"])
    timezone = body.get("timezone", item.get("timezone", "America/Chicago"))

    schedules_table.update_item(
        Key={"schedule_id": schedule_id},
        UpdateExpression="""
            SET platform = :platform,
                content_type = :content_type,
                topic = :topic,
                schedule_expression = :schedule_expression,
                timezone = :timezone,
                updated_at = :updated_at
        """,
        ExpressionAttributeValues={
```

```
            ":platform": platform,
            ":content_type": content_type,
            ":topic": topic,
            ":schedule_expression": schedule_expression,
            ":timezone": timezone,
            ":updated_at": now_iso()
        }
    )

    scheduler.update_schedule(
        Name=schedule_name,
        ScheduleExpression=schedule_expression,
        ScheduleExpressionTimezone=timezone,
        FlexibleTimeWindow={
            "Mode": "OFF"
        },
        Target={
            "Arn": WORKER_LAMBDA_ARN,
            "RoleArn": SCHEDULER_ROLE_ARN,
            "Input": json.dumps({
                "schedule_id": schedule_id
            })
        },
        State="ENABLED"
    )

    return {
        "message": "Schedule updated",
        "schedule_id": schedule_id
    }


def inactive_schedule(body):
    schedule_id = body.get("schedule_id")

    if not schedule_id:
        raise ValueError("schedule_id is required")

    response = schedules_table.get_item(
        Key={"schedule_id": schedule_id}
    )

    item = response.get("Item")

    if not item:
        raise ValueError("Schedule not found")

    schedules_table.update_item(
        Key={"schedule_id": schedule_id},
```

```
        UpdateExpression="SET #status = :status, updated_at = :updated_at",
        ExpressionAttributeNames={
            "#status": "status"
        },
        ExpressionAttributeValues={
            ":status": "inactive",
            ":updated_at": now_iso()
        }
    )

    scheduler.update_schedule(
        Name=item["schedule_name"],
        ScheduleExpression=item["schedule_expression"],
        ScheduleExpressionTimezone=item.get("timezone", "America/Chicago"),
        FlexibleTimeWindow={
            "Mode": "OFF"
        },
        Target={
            "Arn": WORKER_LAMBDA_ARN,
            "RoleArn": SCHEDULER_ROLE_ARN,
            "Input": json.dumps({
                "schedule_id": schedule_id
            })
        },
        State="DISABLED"
    )

    return {
        "message": "Schedule inactivated",
        "schedule_id": schedule_id
    }


def delete_schedule(body):
    schedule_id = body.get("schedule_id")

    if not schedule_id:
        raise ValueError("schedule_id is required")

    response = schedules_table.get_item(
        Key={"schedule_id": schedule_id}
    )

    item = response.get("Item")

    if not item:
        raise ValueError("Schedule not found")

    scheduler.delete_schedule(
```

```
        Name=item["schedule_name"]
    )

    schedules_table.delete_item(
        Key={"schedule_id": schedule_id}
    )

    return {
        "message": "Schedule deleted",
        "schedule_id": schedule_id
    }


def lambda_handler(event, context):
    print("Received event:", json.dumps(event))

    body = event.get("body", event)

    if isinstance(body, str):
        body = json.loads(body)

    action = body.get("action")

    if not action:
        raise ValueError("action is required")

    if action == "connect_social":
        result = create_social_connection(body)

    elif action == "create_schedule":
        result = create_schedule(body)

    elif action == "view_schedule":
        result = view_schedule(body)

    elif action == "update_schedule":
        result = update_schedule(body)

    elif action == "inactive_schedule":
        result = inactive_schedule(body)

    elif action == "delete_schedule":
        result = delete_schedule(body)

    else:
        raise ValueError(f"Unsupported action: {action}")

    return {
```

```
        "statusCode": 200,
        "headers": {
            "Content-Type": "application/json"
        },
        "body": json.dumps(result)
    }
```

Note: The handler reads event.get("body", event) and parses it if it is a string. This one detail makes the same function work for BOTH direct Lambda invokes (payload is the body) and API Gateway HTTP API proxy events (payload arrives as a JSON string under "body"). That is what makes the Amplify + API Gateway integration in Step 7 work without code changes. view_schedule returns the raw DynamoDB item; if a schedule ever stores numeric attributes, add a Decimal-safe JSON encoder before json.dumps.

9.2 Package and deploy

▶ RUN / CODE

```
cd schedule_manager_lambda
zip -r schedule_manager_lambda.zip lambda_function.py

aws lambda create-function \
  --function-name MarketingScheduleManager \
  --runtime python3.12 \
  --role arn:aws:iam::YOUR_ACCOUNT_ID:role/MarketingAutomationLambdaRole \
  --handler lambda_function.lambda_handler \
  --zip-file fileb://schedule_manager_lambda.zip \
  --timeout 60 \
  --memory-size 512 \
  --region us-east-1
cd ..
```

✓ EXPECTED OUTPUT

```
{
    "FunctionName": "MarketingScheduleManager",
    "State": "Pending"   (becomes "Active" within seconds)
}
```

10. Step 7 - Expose the Schedule Manager via API Gateway (HTTP API)

Resources in this step: `MarketingAutomationAPI` `MarketingScheduleManager`

WHAT WE ARE DOING

Create an HTTP API with a Lambda proxy integration to MarketingScheduleManager, enable CORS, and grant API Gateway permission to invoke the function.

WHY

The Amplify-hosted React app runs in the browser and cannot invoke Lambda directly without AWS credentials. API Gateway gives it a public HTTPS endpoint. CORS must allow the Amplify domain (we use * for the lab) or every browser call will fail preflight. The lambda add-permission call is required because API Gateway - not our scheduler role - is now also an invoker of this function.

10.1 Create the HTTP API (quick-create with Lambda target)

▶ RUN / CODE

```
aws apigatewayv2 create-api \
  --name MarketingAutomationAPI \
  --protocol-type HTTP \
  --target arn:aws:lambda:us-east-
1:YOUR_ACCOUNT_ID:function:MarketingScheduleManager \
  --cors-configuration
AllowOrigins="*",AllowMethods="POST,OPTIONS",AllowHeaders="content-type" \
  --region us-east-1
```

✓ EXPECTED OUTPUT

```
{
    "ApiEndpoint": "https://a1b2c3d4e5.execute-api.us-east-1.amazonaws.com",
    "ApiId": "a1b2c3d4e5",
    "Name": "MarketingAutomationAPI",
    "ProtocolType": "HTTP"
}
```

Record the **ApiEndpoint** and **ApiId** - the endpoint is your API_URL for the frontend and integration tests.

10.2 Allow API Gateway to invoke the Lambda

▶ RUN / CODE

```
aws lambda add-permission \
  --function-name MarketingScheduleManager \
  --statement-id apigateway-invoke \
  --action lambda:InvokeFunction \
  --principal apigateway.amazonaws.com \
  --source-arn "arn:aws:execute-api:us-east-1:YOUR_ACCOUNT_ID:YOUR_API_ID/*" \
  --region us-east-1
```

10.3 Smoke-test the API with curl

▶ RUN / CODE

```
curl -s -X POST "https://YOUR_API_ID.execute-api.us-east-1.amazonaws.com/" \
  -H "Content-Type: application/json" \
  -d '{"action": "view_schedule", "schedule_id": "does-not-exist"}'
```

✓ EXPECTED OUTPUT

```
{"message": "Schedule not found"}
```

Note: Quick-create routes ANY method at the root path ($default route) to the Lambda. That is enough for this action-based API. If you prefer REST-style routes (POST /schedules etc.), create explicit routes and integrations instead.

11. Step 8 - Amplify Frontend (React UI)

Resources in this step: `marketing-frontend` `MarketingAutomationAPI`

WHAT WE ARE DOING

Build a small React app with a form to connect a social account and create/view/update/inactivate/delete schedules through the API, then deploy it with AWS Amplify Hosting.

WHY

This is the 'Logged-in User' box from the architecture diagram. Amplify Hosting gives you CDN-backed HTTPS hosting with zero server management, which matches the serverless backend. The app talks only to the API Gateway endpoint - no AWS SDK or credentials ship to the browser.

11.1 Create the React app

▶ **RUN / CODE**

```
cd frontend
npm create vite@latest marketing-frontend -- --template react
cd marketing-frontend
npm install
```

11.2 Configure the API URL - .env

▶ **RUN / CODE**

```
VITE_API_URL=https://YOUR_API_ID.execute-api.us-east-1.amazonaws.com
```

11.3 Replace src/App.jsx

▶ **RUN / CODE**

```
import { useState } from "react";

const API_URL = import.meta.env.VITE_API_URL;

async function callApi(payload) {
  const response = await fetch(API_URL, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(payload)
  });
  return response.json();
}
```

```
export default function App() {
  const [userId] = useState("user123");
  const [platform, setPlatform] = useState("facebook");
  const [contentType, setContentType] = useState("post");
  const [topic, setTopic] = useState("Women apparel Thanksgiving discount");
  const [cron, setCron] = useState("cron(0 9 * * ? *)");
  const [scheduleId, setScheduleId] = useState("");
  const [output, setOutput] = useState("");

  const show = (data) => setOutput(JSON.stringify(data, null, 2));

  const connectSocial = async () => show(await callApi({
    action: "connect_social",
    user_id: userId,
    platform: platform,
    access_token: "TEST_ACCESS_TOKEN",
    page_id: "TEST_PAGE_ID"
  }));

  const createSchedule = async () => {
    const data = await callApi({
      action: "create_schedule",
      user_id: userId,
      platform: platform,
      content_type: contentType,
      topic: topic,
      schedule_expression: cron,
      timezone: "America/Chicago"
    });
    if (data.schedule_id) setScheduleId(data.schedule_id);
    show(data);
  };

  const viewSchedule = async () => show(await callApi({
    action: "view_schedule", schedule_id: scheduleId
  }));

  const updateSchedule = async () => show(await callApi({
    action: "update_schedule", schedule_id: scheduleId,
    topic: topic, content_type: contentType,
    schedule_expression: cron
  }));

  const inactivateSchedule = async () => show(await callApi({
    action: "inactive_schedule", schedule_id: scheduleId
  }));

  const deleteSchedule = async () => show(await callApi({
    action: "delete_schedule", schedule_id: scheduleId
```

```
  }));

  return (
    <div style={{ maxWidth: 640, margin: "40px auto", fontFamily: "sans-serif"
}}>
      <h1>Marketing Automation</h1>

      <h2>1. Connect Social Account</h2>
      <select value={platform} onChange={(e) => setPlatform(e.target.value)}>
        <option value="facebook">Facebook</option>
        <option value="youtube">YouTube</option>
        <option value="linkedin">LinkedIn</option>
      </select>
      <button onClick={connectSocial}>Connect</button>

      <h2>2. Create Schedule</h2>
      <select value={contentType} onChange={(e) =>
setContentType(e.target.value)}>
        <option value="post">Post</option>
        <option value="flyer">Flyer</option>
        <option value="image">Image</option>
        <option value="reel">Reel</option>
      </select>
      <input value={topic} onChange={(e) => setTopic(e.target.value)}
             placeholder="Topic" style={{ width: "100%", margin: "8px 0" }} />
      <input value={cron} onChange={(e) => setCron(e.target.value)}
             placeholder="cron(0 9 * * ? *)" style={{ width: "100%", margin:
"8px 0" }} />
      <button onClick={createSchedule}>Create Schedule</button>

      <h2>3. Manage Schedule</h2>
      <input value={scheduleId} onChange={(e) => setScheduleId(e.target.value)}
             placeholder="schedule_id" style={{ width: "100%", margin: "8px 0"
}} />
      <button onClick={viewSchedule}>View</button>{" "}
      <button onClick={updateSchedule}>Update</button>{" "}
      <button onClick={inactivateSchedule}>Inactivate</button>{" "}
      <button onClick={deleteSchedule}>Delete</button>

      <h2>Result</h2>
      <pre style={{ background: "#f4f4f4", padding: 12 }}>{output}</pre>
    </div>
  );
}
```

11.4 Test locally

▶ **RUN / CODE**

```
npm run dev
```

✓ EXPECTED OUTPUT

```
VITE ready in ... ms
Local: http://localhost:5173/

In the browser: click Connect, then Create Schedule.
The Result panel shows:
{
  "message": "Schedule created",
  "schedule_id": "3f8a...-uuid",
  "schedule_name": "marketing-3f8a...-uuid"
}
```

11.5 Deploy to Amplify Hosting (manual deploy, no Git required)

▶ RUN / CODE

```
npm run build

aws amplify create-app --name marketing-frontend --region us-east-1
# note the appId in the output, e.g. d1abc2def3

aws amplify create-branch --app-id YOUR_APP_ID --branch-name main --region us-
east-1

cd dist && zip -r ../site.zip . && cd ..

aws amplify create-deployment \
  --app-id YOUR_APP_ID --branch-name main --region us-east-1
# note "jobId" and "zipUploadUrl" in the output

curl -T site.zip "PASTE_ZIP_UPLOAD_URL_HERE"

aws amplify start-deployment \
  --app-id YOUR_APP_ID --branch-name main --job-id YOUR_JOB_ID --region us-east-
1
```

✓ EXPECTED OUTPUT

```
{
    "jobSummary": {
        "jobId": "1",
        "status": "PENDING"    (then SUCCEED)
    }
}
```

```
App is live at:
https://main.YOUR_APP_ID.amplifyapp.com
```

Note: If your code is in GitHub, the easier path is the Amplify console: New app -> Host web app -> connect the repo -> build settings are auto-detected for Vite. Every git push then redeploys automatically. Since VITE_API_URL is baked in at build time, set it as an Amplify environment variable in that flow.

11.6 Validate end to end

1. Open https://main.YOUR_APP_ID.amplifyapp.com in a browser.
2. Click Connect - the Result panel shows 'Social connection saved'.
3. Click Create Schedule - the Result panel shows a schedule_id.
4. Run: aws scheduler list-schedules --region us-east-1 - the new marketing-<uuid> schedule is ENABLED.

12. Step 9 - Functional Testing (Lambda and API)

Resources in this step: `MarketingScheduleManager` `MarketingContentWorker` `ScheduleLogs`

WHAT WE ARE DOING

Exercise every action twice - once by invoking the Lambda directly with the AWS CLI, once through the API Gateway endpoint - and verify the results in DynamoDB and EventBridge Scheduler.

WHY

Direct Lambda invokes isolate backend logic from HTTP concerns; API calls prove the full path the Amplify frontend uses (CORS, proxy event shape, permissions). Testing both makes failures easy to localize: if the direct invoke works but the API call fails, the problem is in API Gateway config, not your code.

12.1 Save a social connection

Lambda:

▶ **RUN / CODE**

```
aws lambda invoke \
  --function-name MarketingScheduleManager \
  --cli-binary-format raw-in-base64-out \
  --payload '{
    "action": "connect_social",
    "user_id": "user123",
    "platform": "facebook",
    "access_token": "TEST_ACCESS_TOKEN",
    "page_id": "TEST_PAGE_ID"
  }' \
  response.json --region us-east-1

cat response.json
```

API:

▶ **RUN / CODE**

```
curl -s -X POST "$API_URL" -H "Content-Type: application/json" -d '{
  "action": "connect_social",
  "user_id": "user123",
  "platform": "linkedin",
  "access_token": "TEST_ACCESS_TOKEN"
}'
```

✓ EXPECTED OUTPUT

```
{"statusCode": 200, "headers": {...}, "body": "{\"message\": \"Social connection saved\", \"user_id\": \"user123\", \"platform\": \"facebook\"}"}

API returns just the body:
{"message": "Social connection saved", "user_id": "user123", "platform": "linkedin"}
```

Verify in DynamoDB:

▶ RUN / CODE

```
aws dynamodb get-item \
  --table-name SocialConnections \
  --key '{"user_id": {"S": "user123"}, "platform": {"S": "facebook"}}' \
  --region us-east-1
```

✓ EXPECTED OUTPUT

```
{
    "Item": {
        "user_id": {"S": "user123"},
        "platform": {"S": "facebook"},
        "access_token": {"S": "TEST_ACCESS_TOKEN"},
        "status": {"S": "active"},
        ...
    }
}
```

12.2 Create a recurring schedule (daily 9:00 AM Central)

▶ RUN / CODE

```
aws lambda invoke \
  --function-name MarketingScheduleManager \
  --cli-binary-format raw-in-base64-out \
  --payload '{
    "action": "create_schedule",
    "user_id": "user123",
    "platform": "facebook",
    "content_type": "post",
    "topic": "Women apparel Thanksgiving discount",
    "schedule_expression": "cron(0 9 * * ? *)",
    "timezone": "America/Chicago"
  }' \
  response.json --region us-east-1

cat response.json
aws scheduler list-schedules --region us-east-1
```

✓ EXPECTED OUTPUT

```
{"message": "Schedule created", "schedule_id": "3f8a1c2e-...", "schedule_name":
"marketing-3f8a1c2e-..."}

{
    "Schedules": [
        {
            "Name": "marketing-3f8a1c2e-...",
            "State": "ENABLED",
            ...
        }
    ]
}
```

Save the schedule_id - the remaining tests use it. Export it for convenience:

▶ RUN / CODE

```
export SCHEDULE_ID=PASTE_YOUR_SCHEDULE_ID
```

12.3 Create a one-time test schedule (fires in ~5 minutes)

Use an `at(...)` expression a few minutes in the future so you can watch a real automated run without waiting for the daily cron. The time is interpreted in the given timezone.

▶ RUN / CODE

```
aws lambda invoke \
  --function-name MarketingScheduleManager \
  --cli-binary-format raw-in-base64-out \
  --payload '{
    "action": "create_schedule",
    "user_id": "user123",
    "platform": "linkedin",
    "content_type": "post",
    "topic": "New fashion arrivals",
    "schedule_expression": "at(2026-07-08T15:30:00)",
    "timezone": "America/Chicago"
  }' \
  response.json --region us-east-1
```

✓ EXPECTED OUTPUT

```
{"message": "Schedule created", "schedule_id": "...", "schedule_name":
"marketing-..."}
```

```
After the time passes, CloudWatch shows the Worker ran and
ScheduleLogs contains a "success" entry for this schedule_id.
```

12.4 View a schedule

▶ **RUN / CODE**

```
curl -s -X POST "$API_URL" -H "Content-Type: application/json" \
  -d "{\"action\": \"view_schedule\", \"schedule_id\": \"$SCHEDULE_ID\"}"
```

✓ **EXPECTED OUTPUT**

```
{
  "schedule_id": "3f8a1c2e-...",
  "schedule_name": "marketing-3f8a1c2e-...",
  "user_id": "user123",
  "platform": "facebook",
  "content_type": "post",
  "topic": "Women apparel Thanksgiving discount",
  "schedule_expression": "cron(0 9 * * ? *)",
  "timezone": "America/Chicago",
  "status": "active",
  "last_run_status": "never_run"
}
```

12.5 Update a schedule

▶ **RUN / CODE**

```
curl -s -X POST "$API_URL" -H "Content-Type: application/json" \
  -d "{\"action\": \"update_schedule\", \"schedule_id\": \"$SCHEDULE_ID\",
       \"content_type\": \"image\",
       \"topic\": \"New women fashion image campaign\",
       \"schedule_expression\": \"cron(0 10 * * ? *)\",
       \"timezone\": \"America/Chicago\"}"
```

✓ **EXPECTED OUTPUT**

```
{"message": "Schedule updated", "schedule_id": "3f8a1c2e-..."}

view_schedule now shows content_type=image and the 10:00 cron.
aws scheduler get-schedule --name marketing-$SCHEDULE_ID shows the new
expression.
```

12.6 Manually invoke the Worker Lambda

This simulates EventBridge Scheduler firing, without waiting for the schedule.

▶ RUN / CODE

```
aws lambda invoke \
  --function-name MarketingContentWorker \
  --cli-binary-format raw-in-base64-out \
  --payload "{\"schedule_id\": \"$SCHEDULE_ID\"}" \
  worker_response.json --region us-east-1

cat worker_response.json

aws logs tail /aws/lambda/MarketingContentWorker --since 10m --region us-east-1
```

✓ EXPECTED OUTPUT

```
{"statusCode": 200, "body": "{\"message\": \"Success\", \"schedule_id\":
\"...\", \"platform\": \"facebook\", \"content_type\": \"image\"}"}

Log lines:
Received event: {"schedule_id": "..."}
(with the fake Facebook token the real Graph API call returns HTTP 400 -
the run is logged as "failed"; use platform "linkedin" or "youtube",
which are mocked, to see a clean "success")
```

Note: Facebook posting is the only real HTTP call in the Worker. With the TEST_ACCESS_TOKEN it will fail as expected - that is useful too: verify a 'failed' entry appears in ScheduleLogs and last_run_status becomes 'failed'. For a guaranteed success path, create the schedule with platform linkedin or youtube (mocked).

12.7 Validate ScheduleLogs and schedule status

▶ RUN / CODE

```
aws dynamodb scan --table-name ScheduleLogs --region us-east-1
```

✓ EXPECTED OUTPUT

```
{
    "Items": [
        {
            "log_id": {"S": "..."},
            "schedule_id": {"S": "..."},
            "status": {"S": "success"},   (or "failed" / "skipped")
            "message": {"S": "Content generated and posted successfully"},
            ...
        }
    ],
    "Count": 1
```

```
}
```

12.8 Inactivate a schedule

▶ RUN / CODE

```
curl -s -X POST "$API_URL" -H "Content-Type: application/json" \
  -d "{\"action\": \"inactive_schedule\", \"schedule_id\": \"$SCHEDULE_ID\"}"

aws scheduler get-schedule --name marketing-$SCHEDULE_ID \
  --region us-east-1 --query State
```

✓ EXPECTED OUTPUT

```
{"message": "Schedule inactivated", "schedule_id": "..."}

"DISABLED"

Invoking the Worker with this schedule_id now returns:
"Schedule inactive. Skipped."  and writes a "skipped" log entry.
```

12.9 Delete a schedule

▶ RUN / CODE

```
curl -s -X POST "$API_URL" -H "Content-Type: application/json" \
  -d "{\"action\": \"delete_schedule\", \"schedule_id\": \"$SCHEDULE_ID\"}"

aws scheduler list-schedules --region us-east-1
```

✓ EXPECTED OUTPUT

```
{"message": "Schedule deleted", "schedule_id": "..."}

The marketing-<uuid> schedule no longer appears in the list,
and view_schedule returns {"message": "Schedule not found"}.
```

13. Unit Tests (pytest + moto)

WHAT WE ARE DOING
Write unit tests that run entirely on your machine. moto fakes DynamoDB in memory; unittest.mock patches the EventBridge Scheduler client and the social-posting functions, so no AWS resources or network calls are touched.

WHY
Unit tests catch logic regressions (validation, routing, skip-inactive behavior, log writing) in milliseconds and run in CI without AWS credentials. Mocking the platform posting functions keeps tests deterministic and free.

13.1 Install test dependencies

▶ **RUN / CODE**

```
pip install pytest moto boto3
```

13.2 Create tests/test_unit.py

▶ **RUN / CODE**

```
import json
import os
import sys
import boto3
import pytest
from moto import mock_aws
from unittest.mock import patch, MagicMock

REGION = "us-east-1"

sys.path.insert(0, os.path.join(os.path.dirname(__file__), "..",
"worker_lambda"))
sys.path.insert(0, os.path.join(os.path.dirname(__file__), "..",
"schedule_manager_lambda"))



def create_tables(dynamodb):
    dynamodb.create_table(
        TableName="SocialConnections",
        KeySchema=[{"AttributeName": "user_id", "KeyType": "HASH"},
                   {"AttributeName": "platform", "KeyType": "RANGE"}],
        AttributeDefinitions=[{"AttributeName": "user_id", "AttributeType":
"S"},
                              {"AttributeName": "platform", "AttributeType":
"S"}],
```

```
        BillingMode="PAY_PER_REQUEST")
    dynamodb.create_table(
        TableName="ContentSchedules",
        KeySchema=[{"AttributeName": "schedule_id", "KeyType": "HASH"}],
        AttributeDefinitions=[{"AttributeName": "schedule_id", "AttributeType":
"S"}],
        BillingMode="PAY_PER_REQUEST")
    dynamodb.create_table(
        TableName="ScheduleLogs",
        KeySchema=[{"AttributeName": "log_id", "KeyType": "HASH"}],
        AttributeDefinitions=[{"AttributeName": "log_id", "AttributeType":
"S"}],
        BillingMode="PAY_PER_REQUEST")


@pytest.fixture
def aws_env():
    os.environ["AWS_ACCESS_KEY_ID"] = "testing"
    os.environ["AWS_SECRET_ACCESS_KEY"] = "testing"
    os.environ["AWS_DEFAULT_REGION"] = REGION
    with mock_aws():
        dynamodb = boto3.client("dynamodb", region_name=REGION)
        create_tables(dynamodb)
        # reload modules so their boto3 clients bind to moto
        for mod in ["lambda_function"]:
            if mod in sys.modules:
                del sys.modules[mod]
        yield boto3.resource("dynamodb", region_name=REGION)


# ---------- Worker Lambda: create_content ----------

class TestCreateContent:
    @pytest.fixture(autouse=True)
    def load_worker(self, aws_env):
        import importlib
        import lambda_function as worker
        importlib.reload(worker)
        self.worker = worker

    def test_post_content(self):
        content = self.worker.create_content("post", "summer sale")
        assert content["type"] == "post"
        assert "summer sale" in content["text"]

    def test_flyer_has_media_url(self):
        content = self.worker.create_content("flyer", "sale")
        assert content["media_url"].endswith(".png")
```

```
    def test_reel_has_video_url(self):
        content = self.worker.create_content("reel", "sale")
        assert content["media_url"].endswith(".mp4")

    def test_unsupported_type_raises(self):
        with pytest.raises(ValueError):
            self.worker.create_content("podcast", "sale")


# ---------- Worker Lambda: handler behavior ----------

class TestWorkerHandler:
    @pytest.fixture(autouse=True)
    def setup(self, aws_env):
        import importlib
        import lambda_function as worker
        importlib.reload(worker)
        self.worker = worker
        self.dynamodb = aws_env
        self.schedules = self.dynamodb.Table("ContentSchedules")
        self.connections = self.dynamodb.Table("SocialConnections")
        self.logs = self.dynamodb.Table("ScheduleLogs")

    def seed(self, status="active", platform="linkedin"):
        self.schedules.put_item(Item={
            "schedule_id": "sched-1", "schedule_name": "marketing-sched-1",
            "user_id": "user123", "platform": platform,
            "content_type": "post", "topic": "unit test topic",
            "status": status})
        self.connections.put_item(Item={
            "user_id": "user123", "platform": platform,
            "access_token": "token", "status": "active"})

    def test_missing_schedule_id_raises(self):
        with pytest.raises(ValueError, match="schedule_id is required"):
            self.worker.lambda_handler({}, None)

    def test_inactive_schedule_skipped(self):
        self.seed(status="inactive")
        result = self.worker.lambda_handler({"schedule_id": "sched-1"}, None)
        assert result["body"] == "Schedule inactive. Skipped."
        logs = self.logs.scan()["Items"]
        assert logs[0]["status"] == "skipped"

    def test_success_path_writes_log_and_status(self):
        self.seed(platform="linkedin")  # linkedin poster is a mock, no network
        result = self.worker.lambda_handler({"schedule_id": "sched-1"}, None)
        assert result["statusCode"] == 200
        item = self.schedules.get_item(Key={"schedule_id": "sched-1"})["Item"]
```

```
        assert item["last_run_status"] == "success"
        logs = self.logs.scan()["Items"]
        assert logs[0]["status"] == "success"

    def test_missing_connection_marks_failed(self):
        self.schedules.put_item(Item={
            "schedule_id": "sched-2", "user_id": "ghost",
            "platform": "linkedin", "content_type": "post",
            "status": "active"})
        with pytest.raises(ValueError, match="Social connection not found"):
            self.worker.lambda_handler({"schedule_id": "sched-2"}, None)
        item = self.schedules.get_item(Key={"schedule_id": "sched-2"})["Item"]
        assert item["last_run_status"] == "failed"
        logs = self.logs.scan()["Items"]
        assert logs[0]["status"] == "failed"


# ---------- Schedule Manager Lambda ----------

class TestScheduleManager:
    @pytest.fixture(autouse=True)
    def setup(self, aws_env):
        import importlib
        sys.path.insert(0, os.path.join(
            os.path.dirname(__file__), "..", "schedule_manager_lambda"))
        if "lambda_function" in sys.modules:
            del sys.modules["lambda_function"]
        import lambda_function as manager
        importlib.reload(manager)
        manager.scheduler = MagicMock()   # patch EventBridge Scheduler client
        self.manager = manager
        self.dynamodb = aws_env
        self.schedules = self.dynamodb.Table("ContentSchedules")

    def invoke(self, body):
        return self.manager.lambda_handler(body, None)

    def test_connect_social_requires_token(self):
        with pytest.raises(ValueError, match="access_token is required"):
            self.invoke({"action": "connect_social",
                         "user_id": "u1", "platform": "facebook"})

    def test_create_schedule_writes_item_and_calls_scheduler(self):
        result = self.invoke({
            "action": "create_schedule", "user_id": "u1",
            "platform": "Facebook", "content_type": "Post",
            "topic": "t", "schedule_expression": "cron(0 9 * * ? *)"})
        body = json.loads(result["body"])
        item = self.schedules.get_item(
```

```
            Key={"schedule_id": body["schedule_id"]})["Item"]
        assert item["platform"] == "facebook"      # lowercased
        assert item["status"] == "active"
        self.manager.scheduler.create_schedule.assert_called_once()
        kwargs = self.manager.scheduler.create_schedule.call_args.kwargs
        assert kwargs["FlexibleTimeWindow"] == {"Mode": "OFF"}
        assert kwargs["State"] == "ENABLED"

    def test_api_gateway_string_body_is_parsed(self):
        event = {"body": json.dumps({"action": "view_schedule",
                                     "schedule_id": "nope"})}
        result = self.invoke(event)
        assert json.loads(result["body"])["message"] == "Schedule not found"

    def test_inactive_schedule_disables_eventbridge(self):
        created = json.loads(self.invoke({
            "action": "create_schedule", "user_id": "u1",
            "platform": "linkedin", "content_type": "post",
            "topic": "t", "schedule_expression": "cron(0 9 * * ? *)"})["body"])
        self.invoke({"action": "inactive_schedule",
                     "schedule_id": created["schedule_id"]})
        item = self.schedules.get_item(
            Key={"schedule_id": created["schedule_id"]})["Item"]
        assert item["status"] == "inactive"
        kwargs = self.manager.scheduler.update_schedule.call_args.kwargs
        assert kwargs["State"] == "DISABLED"

    def test_delete_schedule_removes_both(self):
        created = json.loads(self.invoke({
            "action": "create_schedule", "user_id": "u1",
            "platform": "linkedin", "content_type": "post",
            "topic": "t", "schedule_expression": "cron(0 9 * * ? *)"})["body"])
        self.invoke({"action": "delete_schedule",
                     "schedule_id": created["schedule_id"]})
        assert "Item" not in self.schedules.get_item(
            Key={"schedule_id": created["schedule_id"]})
        self.manager.scheduler.delete_schedule.assert_called_once()

    def test_unsupported_action_raises(self):
        with pytest.raises(ValueError, match="Unsupported action"):
            self.invoke({"action": "explode"})
```

Note: Both Lambda folders contain a file named lambda_function.py, so the test file manipulates sys.path and reloads the module per test class. Reloading inside the moto context is what makes the module-level boto3 clients bind to the fake DynamoDB.

13.3 Run

▶ RUN / CODE

```
cd aws-marketing-automation
python -m pytest tests/test_unit.py -v
```

✓ EXPECTED OUTPUT

```
tests/test_unit.py::TestCreateContent::test_post_content PASSED
tests/test_unit.py::TestCreateContent::test_flyer_has_media_url PASSED
tests/test_unit.py::TestCreateContent::test_reel_has_video_url PASSED
tests/test_unit.py::TestCreateContent::test_unsupported_type_raises PASSED
tests/test_unit.py::TestWorkerHandler::test_missing_schedule_id_raises PASSED
tests/test_unit.py::TestWorkerHandler::test_inactive_schedule_skipped PASSED
tests/test_unit.py::TestWorkerHandler::test_success_path_writes_log_and_status
PASSED
tests/test_unit.py::TestWorkerHandler::test_missing_connection_marks_failed
PASSED
tests/test_unit.py::TestScheduleManager::test_connect_social_requires_token
PASSED
tests/test_unit.py::TestScheduleManager::test_create_schedule_writes_item_and_ca
lls_scheduler PASSED
tests/test_unit.py::TestScheduleManager::test_api_gateway_string_body_is_parsed
PASSED
tests/test_unit.py::TestScheduleManager::test_inactive_schedule_disables_eventbr
idge PASSED
tests/test_unit.py::TestScheduleManager::test_delete_schedule_removes_both
PASSED
tests/test_unit.py::TestScheduleManager::test_unsupported_action_raises PASSED

============ 14 passed in ~5s ============
```

14. Integration Tests (Live AWS via API Gateway and Lambda)

WHAT WE ARE DOING

Run an end-to-end suite against the deployed stack: connect a social account through the API, create a schedule, verify the EventBridge schedule exists, run the Worker via a real Lambda invoke, confirm a success log lands in ScheduleLogs, then update, inactivate, and delete - verifying each transition in both DynamoDB and EventBridge Scheduler.

WHY

Unit tests can't prove IAM permissions, API Gateway routing/CORS, the Lambda proxy event shape, or the DynamoDB-to-EventBridge sync actually work in your account. This suite exercises the exact path the Amplify frontend and the scheduler use in production. It uses the mocked linkedin platform so no real social post is made.

14.1 Create tests/test_integration.py

▶ **RUN / CODE**

```
import json
import os
import time
import boto3
import pytest
import requests

API_URL = os.environ["API_URL"]          # e.g. https://a1b2c3.execute-api.us-
east-1.amazonaws.com
REGION = "us-east-1"

lambda_client = boto3.client("lambda", region_name=REGION)
scheduler = boto3.client("scheduler", region_name=REGION)
dynamodb = boto3.resource("dynamodb", region_name=REGION)
logs_table = dynamodb.Table("ScheduleLogs")



def call_api(payload):
    response = requests.post(API_URL, json=payload, timeout=30)
    assert response.status_code == 200, response.text
    return response.json()



@pytest.fixture(scope="module")
def schedule_id():
    # setup: connect account + create schedule through the API
```

```
    result = call_api({
        "action": "connect_social",
        "user_id": "itest-user",
        "platform": "linkedin",
        "access_token": "ITEST_TOKEN"
    })
    assert result["message"] == "Social connection saved"

    result = call_api({
        "action": "create_schedule",
        "user_id": "itest-user",
        "platform": "linkedin",
        "content_type": "post",
        "topic": "integration test topic",
        "schedule_expression": "cron(0 9 * * ? *)",
        "timezone": "America/Chicago"
    })
    sid = result["schedule_id"]

    yield sid

    # teardown: delete if the last test didn't
    try:
        call_api({"action": "delete_schedule", "schedule_id": sid})
    except Exception:
        pass


def test_01_schedule_visible_via_api(schedule_id):
    item = call_api({"action": "view_schedule", "schedule_id": schedule_id})
    assert item["status"] == "active"
    assert item["platform"] == "linkedin"


def test_02_eventbridge_schedule_created(schedule_id):
    schedule = scheduler.get_schedule(Name=f"marketing-{schedule_id}")
    assert schedule["State"] == "ENABLED"
    assert schedule["FlexibleTimeWindow"]["Mode"] == "OFF"
    target_input = json.loads(schedule["Target"]["Input"])
    assert target_input["schedule_id"] == schedule_id


def test_03_worker_lambda_posts_and_logs(schedule_id):
    response = lambda_client.invoke(
        FunctionName="MarketingContentWorker",
        Payload=json.dumps({"schedule_id": schedule_id}))
    payload = json.loads(response["Payload"].read())
    assert response["StatusCode"] == 200
    assert "FunctionError" not in response
```

```
    body = json.loads(payload["body"])
    assert body["message"] == "Success"

    time.sleep(2)
    logs = [i for i in logs_table.scan()["Items"]
            if i["schedule_id"] == schedule_id]
    assert any(l["status"] == "success" for l in logs)

    item = call_api({"action": "view_schedule", "schedule_id": schedule_id})
    assert item["last_run_status"] == "success"


def test_04_update_schedule_via_api(schedule_id):
    call_api({
        "action": "update_schedule",
        "schedule_id": schedule_id,
        "topic": "updated integration topic",
        "schedule_expression": "cron(0 10 * * ? *)"
    })
    item = call_api({"action": "view_schedule", "schedule_id": schedule_id})
    assert item["topic"] == "updated integration topic"
    schedule = scheduler.get_schedule(Name=f"marketing-{schedule_id}")
    assert schedule["ScheduleExpression"] == "cron(0 10 * * ? *)"


def test_05_inactivate_then_worker_skips(schedule_id):
    call_api({"action": "inactive_schedule", "schedule_id": schedule_id})

    schedule = scheduler.get_schedule(Name=f"marketing-{schedule_id}")
    assert schedule["State"] == "DISABLED"

    response = lambda_client.invoke(
        FunctionName="MarketingContentWorker",
        Payload=json.dumps({"schedule_id": schedule_id}))
    payload = json.loads(response["Payload"].read())
    assert payload["body"] == "Schedule inactive. Skipped."


def test_06_delete_schedule(schedule_id):
    call_api({"action": "delete_schedule", "schedule_id": schedule_id})

    item = call_api({"action": "view_schedule", "schedule_id": schedule_id})
    assert item["message"] == "Schedule not found"

    with pytest.raises(scheduler.exceptions.ResourceNotFoundException):
        scheduler.get_schedule(Name=f"marketing-{schedule_id}")
```

14.2 Run

▶ RUN / CODE

```
pip install requests
export API_URL=https://YOUR_API_ID.execute-api.us-east-1.amazonaws.com
python -m pytest tests/test_integration.py -v
```

✓ EXPECTED OUTPUT

```
tests/test_integration.py::test_01_schedule_visible_via_api PASSED
tests/test_integration.py::test_02_eventbridge_schedule_created PASSED
tests/test_integration.py::test_03_worker_lambda_posts_and_logs PASSED
tests/test_integration.py::test_04_update_schedule_via_api PASSED
tests/test_integration.py::test_05_inactivate_then_worker_skips PASSED
tests/test_integration.py::test_06_delete_schedule PASSED

============ 6 passed in ~20s ============
```

Note: Integration tests need live AWS credentials with access to Lambda, Scheduler, and DynamoDB, and they create/delete real resources named with the itest-user prefix. Run them against a dev account, never production.

15. Final Validation Checklist

Check	Command	Expect
3 DynamoDB tables exist	aws dynamodb list-tables --region us-east-1	SocialConnections, ContentSchedules, ScheduleLogs
Both Lambdas deployed	aws lambda list-functions --region us-east-1	MarketingScheduleManager, MarketingContentWorker
Schedule exists and enabled	aws scheduler list-schedules --region us-east-1	marketing-<uuid>, ENABLED
API reachable	curl -X POST $API_URL -d '{"action":"view_schedule","schedule_id":"x"}'	Schedule not found
Amplify app live	open https://main.<appId>.amplifyapp.com	UI loads, buttons work
Worker logs clean	aws logs tail /aws/lambda/MarketingContentWorker --since 1h	Received event / Success
Run history recorded	aws dynamodb scan --table-name ScheduleLogs	status = success
Unit tests green	python -m pytest tests/test_unit.py -v	14 passed
Integration tests green	python -m pytest tests/test_integration.py -v	6 passed

16. Security Notes (Read Before Production)

- **Tokens:** do not store raw OAuth access tokens in DynamoDB for production. Store them in AWS Secrets Manager (best) or encrypt the DynamoDB table with a customer-managed KMS key (acceptable), and keep only a credential reference in the table. Implement token refresh, and never return tokens in any API response.
- **CORS:** replace AllowOrigins="*" with your exact Amplify domain (https://main.<appId>.amplifyapp.com) once deployed.
- **Auth:** this lab's API is public. Add an Amazon Cognito user pool as an API Gateway JWT authorizer so 'Logged-in User' is real - Amplify's Auth library integrates with Cognito directly, and user_id should come from the JWT, not the request body.
- **IAM:** the scheduler:* actions currently use Resource:'*'; scope them to arn:aws:scheduler:us-east-1:ACCOUNT:schedule/default/marketing-* in production.

17. Cleanup

To avoid charges after the lab:

▶ **RUN / CODE**

```
# delete any remaining schedules
aws scheduler list-schedules --query "Schedules[].Name" --output text --region
us-east-1
aws scheduler delete-schedule --name marketing-YOUR_UUID --region us-east-1

aws lambda delete-function --function-name MarketingContentWorker --region us-
east-1
aws lambda delete-function --function-name MarketingScheduleManager --region us-
east-1

aws apigatewayv2 delete-api --api-id YOUR_API_ID --region us-east-1
aws amplify delete-app --app-id YOUR_APP_ID --region us-east-1

aws dynamodb delete-table --table-name SocialConnections --region us-east-1
aws dynamodb delete-table --table-name ContentSchedules --region us-east-1
aws dynamodb delete-table --table-name ScheduleLogs --region us-east-1

aws iam delete-role-policy --role-name MarketingAutomationLambdaRole --policy-
name MarketingAutomationLambdaPolicy
aws iam detach-role-policy --role-name MarketingAutomationLambdaRole --policy-
arn arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole
aws iam delete-role --role-name MarketingAutomationLambdaRole
aws iam delete-role-policy --role-name EventBridgeSchedulerInvokeLambdaRole --
policy-name SchedulerInvokeLambdaPolicy
aws iam delete-role --role-name EventBridgeSchedulerInvokeLambdaRole
```

18. Lab Complete - End-to-End Result

If every checklist item in Section 15 passed, you now have a fully working, serverless marketing automation platform:

- A user opens your Amplify site at `https://main.<appId>.amplifyapp.com`, connects a social account, and creates a schedule from the browser.
- The request travels through `MarketingAutomationAPI` to **`MarketingScheduleManager`**, which stores the profile in **`ContentSchedules`** and arms an EventBridge schedule with exact-time delivery.
- At the scheduled moment, EventBridge Scheduler fires **`MarketingContentWorker`**, which generates the post / flyer / image / reel and publishes it to Facebook, YouTube, or LinkedIn.
- Every run is recorded in **`ScheduleLogs`** with success / failed / skipped status, and the schedule's last_run_status is visible in the UI via view_schedule.
- 14 unit tests and 6 integration tests prove the logic and the deployed stack, and can run in CI on every change.

Next steps: swap the mocked LinkedIn/YouTube posters for real API integrations, plug Amazon Bedrock into create_content, add Cognito authentication, and move tokens to Secrets Manager (Section 16).